

Лекция 2

1. Прогон программы вывода строки в формате Unicode.

Код:

```
✓ #include <windows.h>
  | #include <tchar.h>
  | #include <stdio.h>
  | #define UNICODE
✓ #ifdef UNICODE
  | #define _UNICODE
  | #endif
  |
✓ int main() {
  |     const WCHAR* TextString = _T("Hello, word");
  |     _tprintf(_T("String - %s\n"), TextString);
  |
  |     return 0;
  | }
```

Результат:

```
String - Hello, word

C:\Users\Анастасия\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe
кодом 0.
```

2. Прогон программы записи в файл в Unicode и обычном формате.

Код:

```

#define UNICODE
#ifdef UNICODE
#define _UNICODE
#endif
#include <windows.h>
#include <tchar.h>
#include <stdio.h>

void main() {
    HANDLE hFile;
    LPCWSTR FileName = _T("MyFile.txt");
    const WCHAR* TextString = _T("Hello, world.");
    DWORD iWrite, StringLength = lstrlen(TextString);

    _tprintf(_T("There are %ld symbols in text string %s\n"), StringLength, TextString);

    hFile = CreateFile(FileName, GENERIC_WRITE, 0, NULL, CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL | 0, NULL);

    iWrite = StringLength;
#ifdef UNICODE
    iWrite = 2 * StringLength;
#endif

    WriteFile(hFile, TextString, iWrite, &iWrite, NULL);
    printf("%d bytes are written to file\n.", iWrite);
    CloseHandle(hFile);
}

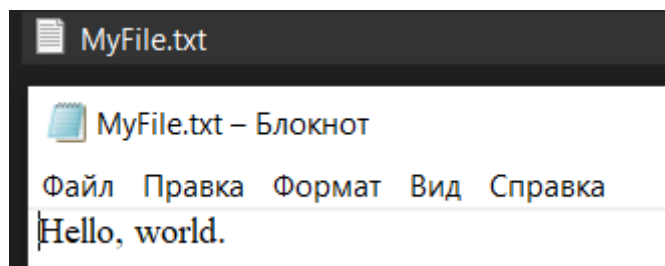
```

Результат:

```

There are 13 symbols in text string Hello, world.
26 bytes are written to file
.
C:\Users\Анастасия\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe
кодом 0.

```



3. Прогон программы, синтезирующей информацию об ошибке, которая имитирует отсутствие нужного файла.

Код:

```

#include <windows.h>
#include <stdio.h>

void GetError() {
    DWORD dw = GetLastError();
    printf("GetLastError returned %u\n", dw);
}

void SetError() {
    DWORD dw = ERROR_FILE_NOT_FOUND;
    SetLastError(dw);
}

void main() {
    SetError();
    GetError();
}

```

Результат:

```

GetLastError returned 2

C:\Users\Анастасия\source\repos\ConsoleApplication2\x64\Debug\ConsoleApplication2.exe
кодом 0.

```

Лекция 3

4. Прогон программы реализующей структурную обработку исключений.

Код:

```

#include <stdio.h>

void main() {
    int i = 1, j = 0, k = 0;
    __try {
        k = i / j;
        puts("in try");
        printf("k = %d\n", k);
    }
    __except (1) {
        puts("in except");
        printf("k=%d\n", k);
    }
}

```

Результат:

```

in except
k=0

C:\Users\Анастасия\source\repos\ConsoleApplication2\x64\Debug\0
кодом 0.

```

Лекция 4. Выдает ошибку.

5. Прогон программы, создающей объект.

Код:

```

#include <windows.h>
#include <stdio.h>
void main(void) {
    HANDLE hMapFile;
    HANDLE hFile;

    hFile = CreateFile("MyFile.txt", GENERIC_READ | GENERIC_WRITE, FILE_SHARE_READ, NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);

    hMapFile = CreateFileMapping(hFile, NULL, PAGE_READWRITE, 0, 0, "MyFileObject");
    getchar();
}

```

Результат:

abc	E0167	аргумент типа "const char *" несовместим с параметром типа "LPCWSTR"	ConsoleApplication2	ConsoleApplication2.cpp	7
abc	E0167	аргумент типа "const char *" несовместим с параметром типа "LPCWSTR"	ConsoleApplication2	ConsoleApplication2.cpp	9
⚠	C4326	возвращаемый тип "main" должен быть "int", а не "void"	ConsoleApplication2	ConsoleApplication2.cpp	3
✖	C2664	"HANDLE CreateFileW (LPCWSTR,DWORD,DWORD,LPSECURIT Y_ATTRIBUTES,DWORD,DWORD,HAND LE)": невозможно преобразовать аргумент 1 из "const char [11]" в "LPCWSTR"	ConsoleApplication2	ConsoleApplication2.cpp	7
✖	C2664	"HANDLE CreateFileMappingW (HANDLE,LPSECURITY_ATTRIBUTES,DW ORD,DWORD,DWORD,LPCWSTR)": невозможно преобразовать аргумент 6 из "const char [13]" в "LPCWSTR"	ConsoleApplication2	ConsoleApplication2.cpp	9

Лекция 5

6. Прогон программы создания процесса.

Код:

```

#include <windows.h>
#include <stdio.h>
void main(VOID) {
    STARTUPINFO StartupInfo;
    PROCESS_INFORMATION ProcInfo;
    TCHAR CommandLine[] = TEXT("sleep");

    ZeroMemory(&StartupInfo, sizeof(StartupInfo));
    StartupInfo.cb = sizeof(StartupInfo);
    ZeroMemory(&ProcInfo, sizeof(ProcInfo));

    if (!CreateProcess(NULL, // Не используется имя модуля
        CommandLine, // Командная строка
        NULL, // Дескриптор процесса не наследуется.
        NULL, // Дескриптор потока не наследуется.
        FALSE, // Установка описателей наследования
        0, // Нет флагов создания процесса
        NULL, // Блок переменных окружения родительского процесса
        NULL, // Использовать текущий каталог родительского процесса
        &StartupInfo, // Указатель на структуру STARTUPINFO.
        &ProcInfo) // Указатель на структуру информации о процессе.
    )

        printf("CreateProcess failed.");

    // Ждать окончания дочернего процесса
    WaitForSingleObject(ProcInfo.hProcess, INFINITE);

    // Закрыть описатели процесса и потока
    CloseHandle(ProcInfo.hProcess);
    CloseHandle(ProcInfo.hThread);
}

```

Результат:

```

CreateProcess failed.
C:\Users\Анастасия\source\repos\ConsoleApplication3\x64\Debug
одом 0.

```

7. Код:

```

#include <windows.h>
#include <stdio.h>
void main(VOID) {
    printf("Данная программа будет спать в течение 10000 мс\n");
    Sleep(10000);
}

```

Результат:

Данная программа будет спать в течение 10000 мс

Данная программа будет спать в течение 10000 мс

C:\Users\Анастасия\source\repos\ConsoleApplication3\x64
кодом 0.

8. Прогон программы, иллюстрирующей состояния потоков.

Код:

```
#include <windows.h>
#include <stdio.h>
#include <math.h>
VOID main(VOID) {
    int i, N = 50000000;
    double a, b;
    getchar();
    printf("Before circle\n");
    for (i = 0; i < N; i++) {
        b = (double)i / (double)N;
        a = sin(b);
    }
    printf("After circle\n");
    getchar();
}
```

Результат:

```
1
Before circle
After circle

C:\Users\Анастасия\source\repos\ConsoleApplication3\x64
одом 0.
```

9. Прогон программы создания потока

Код:

```

#include <windows.h>
#include <stdio.h>
DWORD WINAPI MyThread(LPVOID lpParam)
{
    printf("Parameter = %d\n", *(DWORD*)lpParam);
    return 0;
}
VOID main(VOID)
{
    DWORD ThreadId, ThreadParameter = 10;
    HANDLE hThread;
    hThread = CreateThread(
        NULL, // атрибуты безопасности по умолчанию
        0, // размер стека по умолчанию
        MyThread, // указатель на процедуру создаваемого потока
        &ThreadParameter, // аргумент, передаваемый функции потока
        0, // флаги создания по умолчанию
        &ThreadId); // возвращаемый идентификатор потока
    if (hThread == NULL) printf("CreateThread failed.");
    getchar();
    CloseHandle(hThread);
}

```

Результат:

```

Parameter = 10
1
C:\Users\Анастасия\source\repos\ConsoleApp1\ConsoleApp1\
одем 0.

```

Лекция 6

10. Прогон программы, демонстрация приоритетного планирования

Код:

```

#include <windows.h>
#include <stdio.h>
#include <math.h>
void Calculations()
{
    int i, N = 50000000;
    double a, b;
    for (i = 0; i < N; i++) {
        b = (double)i / (double)N;
        a = sin(b);
    }
}

DWORD WINAPI SecondThread(LPVOID lpParam)
{
    printf("Begin of Second Thread\n");
    Calculations();
    printf("End of Second Thread\n");
    return 0;
}

VOID main(VOID)
{
    DWORD dwThreadId, dwThrdParam;
    HANDLE hThread;
    hThread = CreateThread(
        NULL,
        0,
        SecondThread,
        &dwThrdParam,
        0,
        &dwThreadId);
    if (hThread == NULL)
    {
        printf("CreateThread failed\n");
        return;
    }
}

```

```

    SetThreadPriority(hThread, THREAD_PRIORITY_ABOVE_NORMAL);
    SuspendThread(hThread);
    getchar();
    ResumeThread(hThread);
    printf("Begin of First Thread\n");
    Calculations();
    printf("End of First Thread\n");
}

```

Результат:

```
2
Begin of First Thread
Begin of Second Thread
End of First Thread
End of Second Thread

C:\Users\Анастасия\source
одом 0.
```

Лекция 7 Выдает ошибку.

11. Прогон программы общения процесса через анонимный канал с самим собой.

Код:

```
✓ #include <windows.h>
  #include <stdio.h>
✓ int main()
  {
    HANDLE hRead, hWrite;
    char BufIn[100], * BufOut = "0123456789";
    int BufSize = 100;
    int BytesOut = 10, BytesIn = 5, i;
    if (!CreatePipe(&hRead, &hWrite, NULL, BufSize))
        printf("Create pipe failed.\n");
    WriteFile(hWrite, BufOut, BytesOut, &BytesOut, NULL);
    printf("Write into pipe %d bytes : ", BytesOut);
    for (i = 0; i < BytesOut; i++) printf("%c", BufOut[i]);
    printf("\n");
    ReadFile(hRead, BufIn, BytesIn, &BytesIn, NULL);
    printf("Read from pipe %d bytes : ", BytesIn);
    for (i = 0; i < 5; i++) printf("%c", BufIn[i]);
    return 0;
  }
```

Результат:

	E0144	значение типа "const char *" нельзя использовать для инициализации сущности типа "char *"	ConsoleApplication3	ConsoleApplication3.cpp	6
	E0167	аргумент типа "int *" несовместим с параметром типа "LPDWORD"	ConsoleApplication3	ConsoleApplication3.cpp	11
	E0167	аргумент типа "int *" несовместим с параметром типа "LPDWORD"	ConsoleApplication3	ConsoleApplication3.cpp	15
	C6001	Использование неинициализированной памяти "dwThrdParam".	ConsoleApplication3	ConsoleApplication3.cpp	28
	C6031	Возвращаемое значение пропущено: "getchar".	ConsoleApplication3	ConsoleApplication3.cpp	38
	C2440	инициализация: невозможно преобразовать "const char [11]" в "char *"	ConsoleApplication3	ConsoleApplication3.cpp	6
	C2664	"BOOL WriteFile (HANDLE,LPCVOID,DWORD,LPDWORD,LPOVERLAPPED)": невозможно преобразовать аргумент 4 из "int *" в "LPDWORD"	ConsoleApplication3	ConsoleApplication3.cpp	11

12. Прогон программы общения двух процессов через именованный канал.

Код:

```

#include <stdio.h>
#include <windows.h>
void main()
{
    PROCESS_INFORMATION piProcInfo;
    STARTUPINFO SI;
    char* ClientName = "client.exe";
    HANDLE hPipe;
    LPTSTR PipeName = TEXT("\\\\.\\pipe\\MyPipe");
    char Buff[255];
    DWORD iNumBytesToRead = 255, i;
    ZeroMemory(&SI, sizeof(STARTUPINFO));
    SI.cb = sizeof(STARTUPINFO);
    ZeroMemory(&piProcInfo, sizeof(piProcInfo));
    hPipe = CreateNamedPipe(
        PipeName, // имя канала
        PIPE_ACCESS_DUPLEX, // чтение и запись из канала
        PIPE_TYPE_MESSAGE | // передача сообщений по каналу
        PIPE_READMODE_MESSAGE | // режим чтения сообщений
        PIPE_WAIT, // синхронная передача сообщений
        PIPE_UNLIMITED_INSTANCES, // число экземпляров канала
        4096, // размер выходного буфера
        4096, // размер входного буфера
        NMPWAIT_USE_DEFAULT_WAIT, // тайм-аут клиента
        NULL); // защита по умолчанию
    if (hPipe == INVALID_HANDLE_VALUE)
    {
        printf("CreatePipe failed: error code %d\n", (int)GetLastError());
        return;
    }
    if ((CreateProcess(NULL, ClientName, NULL, NULL, FALSE, 0, NULL,
        NULL, &SI, &piProcInfo)) == 0)
    {
        printf("create client process: error code %d\n", (int)GetLastError());
        return;
    }
}

```

```

    if ((ConnectNamedPipe(hPipe, NULL)) == 0)
    {
        printf("client could not connect\n");
        return;
    }
    ReadFile(hPipe, Buff, iNumBytesToRead, &iNumBytesToRead, NULL);
    for (i = 0; i < iNumBytesToRead; i++) printf("%c", Buff[i]);
}

#include <stdio.h>
#include <windows.h>
void main()
{
    HANDLE hPipe;
    LPTSTR PipeName = TEXT("\\\\.\\pipe\\MyPipe");
    DWORD NumBytesToWrite;
    char Buff[] = "Message from Client";
    hPipe = CreateFile(
        PipeName, // имя канала
        GENERIC_READ | // чтение и запись в канал
        GENERIC_WRITE,
        0, // нет разделяемых операций
        NULL, // защита по умолчанию
        OPEN_EXISTING, // открытие существующего канала
        0, // атрибуты по умолчанию
        NULL); // нет дополнительных атрибутов
    WriteFile(hPipe, Buff, strlen(Buff), &NumBytesToWrite, NULL);
}

```

Результат:

 E0144	значение типа "const char *" нельзя использовать для инициализации сущности типа "char *"	ConsoleApplication3	ConsoleApplication3.cpp	7
 E0144	значение типа "const wchar_t *" нельзя использовать для инициализации сущности типа "LPTSTR"	ConsoleApplication3	ConsoleApplication3.cpp	9
 E0167	аргумент типа "char *" несовместим с параметром типа "LPWSTR"	ConsoleApplication3	ConsoleApplication3.cpp	31
 E0144	значение типа "const wchar_t *" нельзя использовать для инициализации сущности типа "LPTSTR"	ConsoleApplication3	ConsoleApplication3.cpp	53
 C6001	Использование неинициализированной памяти "dwThrdParam".	ConsoleApplication3	ConsoleApplication3.cpp	28